



JMPM

# Manual do Usuário - AdvPlc (Compilador AdvPL/TLPP)

---

DATA

16/09/2026

ARQUIVO

MANUAL\_ADVPLC.md



## Introdução

O AdvPlc é o compilador oficial do AdvPP para as linguagens AdvPL e TLPP. Ele processa código fonte e gera bytecode executável pela máquina virtual do AdvPP, suportando tanto a sintaxe clássica do AdvPL quanto as extensões modernas do TLPP.

## Requisitos do Sistema

- **Sistema Operacional:** Linux, Windows 64-bit ou macOS (binário 100% estático, sem dependências externas)
- **Arquiteturas:** amd64 e arm64
- **Espaço em Disco:** ~15MB

## Instalação

### Via Release do GitHub (recomendado)

Baixe o pacote da sua plataforma em <https://github.com/peder1981/AdvPP/releases>:

- `advpp-cli-<versão>-linux-amd64.tar.gz` / `linux-arm64`
- `advpp-cli-<versão>-windows-amd64.zip`
- `advpp-cli-<versão>-darwin-arm64.tar.gz` (macOS Apple Silicon)
- `advpp_<versão>_amd64.deb` (Debian/Ubuntu, inclui as ferramentas gráficas)

```
# Linux/macOS
tar xzf advpp-cli-*.tar.gz && sudo mv advplc /usr/local/bin/

# Debian/Ubuntu (suite completa)
sudo dpkg -i advpp-*_amd64.deb
```

### Via Compilação

```
# Clonar repositório
git clone https://github.com/peder1981/AdvPP.git
cd AdvPP

# Compilar todas as ferramentas
make build

# Cross-compilar o CLI para todas as plataformas (dist/)
make cross
```

## Visão Geral

### O que é o AdvPlc?

O AdvPlc é um compilador que:

- **Processa código fonte** AdvPL/TLPP
- **Gera bytecode** para a máquina virtual
- **Valida sintaxe** e semântica
- **Otimiza código** para melhor performance
- **Gera informações** de debug

### Fluxo de Compilação

```
Código Fonte (.prw/.tlpp)
↓
Análise Léxica (Tokenização)
↓
Análise Sintática (AST)
```

```
↓
Análise Semântica (Validação)
↓
Otimização
↓
Geração de Bytecode (.bytecode)
↓
Execução na VM
```

## Uso Básico

O advplc trabalha com subcomandos:

```
advplc <comando> <arquivo(s)> [opções]
```

### Executar um fonte diretamente

```
advplc run programa.prw
advplc run programa.prw --ui           # com diálogos gráficos (Fyne)
advplc run programa.prw --db-path meu.db # conectado a um banco específico
```

### Compilar para bytecode

```
advplc compile programa.prw -o programa.bytecode
advplc exec programa.bytecode           # executa o bytecode
```

### Verificar sintaxe (um ou vários arquivos, em paralelo)

```
advplc check programa.prw

# Múltiplos arquivos: verificação paralela (1 worker por CPU).
# Os arquivos vêm ANTES das opções:
advplc check src/*.prw src/*.tlpp -I ./includes
```

Saída do modo múltiplo: `OK / FAIL` por arquivo e um resumo  
( `checked N files: X ok, Y failed (W workers)` ). Código de saída 1 se  
qualquer arquivo falhar.

### Executável standalone

```
advplc build programa.prw -o programa
```

Embuta bytecode e runtime num executável independente (requer Go instalado,  
ou a variável `ADVPP_SRC` apontando para um checkout deste repositório).

O binário resultante detecta sozinho se deve rodar em console ou abrir uma  
janela **Fyne** (GUI desktop nativa, não é PO-UI/web): varre o bytecode em  
busca de UI natives ( `FWGetText` , `FWMenuSelect` ), instanciação de  
componentes ( `FWMBrowse` , `MSDIALOG` e afins) ou anotações REST  
( `@Get` / `@Post` /etc). Se nada disso for encontrado, roda sempre em console  
(mesmo sem terminal). Se algo for encontrado e stdin não for um  
terminal real (TTY), abre a janela Fyne; com TTY, roda em console.

```
advplc build programa.prw -o programa --gui
```

`--gui` fixa o programa como app desktop **em tempo de build**: a  
alternativa de console nunca é tentada, e no Windows o binário é linkado no  
subsistema GUI ( `-H=windowsgui` ) — nenhuma janela de console aparece, nem

por um instante. **Obrigatório para qualquer programa que use** `MSDIALOG` e recomendado para qualquer app pensado como desktop (ex.: e-Gov, GesCon). Para forçar a GUI num binário já compilado sem essa flag, sem recompilar, use a variável de ambiente `ADVPP_FORCE_GUI=1 ./programa` em tempo de execução — mais fraca que `--gui` (não muda o subsistema Windows).

Modo web (renderer no browser)

```
advplc serve programa.prw           # http://localhost:8080
advplc serve programa.prw --port 9000
advplc serve programa.prw --watch   # hot reload ao salvar o fonte
```

Executa o programa no servidor (mesma VM, mesmo banco `ADVPP.db`) e renderiza a interface no browser com **PO-UI** (framework visual da TOTVS), embutida no próprio binário — não precisa de Node, npm ou SmartClient. Cada aba/recarga do browser cria uma sessão com VM isolada e conexão própria ao banco.

O que é renderizado:

| Recurso AdvPL                                                                                                              | No browser                                                                                                                                                                                     |
|----------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ConOut(...)</code>                                                                                                   | Console em tempo real                                                                                                                                                                          |
| <code>MsgInfo</code> / <code>MsgStop</code> / <code>MsgAlert</code> / <code>MsgYesNo</code> / <code>Alert</code>           | Diálogos PO-UI que bloqueiam a execução até a resposta                                                                                                                                         |
| <code>FWMBrowse():New()</code> + <code>SetAlias("SA1")</code> + <code>SetDescription(...)</code> + <code>Activate()</code> | <code>po-table</code> com colunas e títulos do dicionário SX3; Incluir/Editar abrem <code>po-dynamic-form</code> gerado do dicionário; Excluir faz soft-delete ( <code>D_E_L_E_T_='*'</code> ) |
| <code>DEFINE MSDIALOG</code> + <code>@ linha,coluna</code><br><code>SAY/GET/BUTTON</code> + <code>ACTIVATE MSDIALOG</code> | Modal PO-UI montado por heurística de grade; o que o usuário digita nos <code>GET</code> s <b>volta para as variáveis</b> do programa                                                          |

Com `--watch` (ou `-w`), o fonte é recompilado a cada alteração e todas as sessões do browser recarregam automaticamente; erro de compilação aparece no console do browser sem recarregar.

A porta padrão pode ser fixada em `~/.advpp/advpp_config.json` (`"webui_port": "9000"`) — configuração compartilhada entre todas as ferramentas AdvPP.

Limitações atuais do MSDIALOG web: codeblocks não capturam variáveis locais (`ACTION {|| oDlg:End()}`) não fecha o diálogo — qualquer clique de botão fecha após executar o `ACTION`); `VALID` por campo ainda não dispara validação no servidor.

Inspeção

```
advplc ast programa.prw           # imprime a AST
advplc bytecode programa.prw      # imprime o bytecode
advplc version                     # versão do compilador
```

Opções de Linha de Comando

| Opção                                                  | Descrição                                 | Exemplo                    |
|--------------------------------------------------------|-------------------------------------------|----------------------------|
| <code>-I, --include</code><br><code>&lt;dir&gt;</code> | Adiciona diretório de include (repetível) | <code>-I ./includes</code> |
| <code>-D, --define</code><br><code>&lt;k=v&gt;</code>  | Define símbolo do preprocessador          | <code>-D DEBUG=1</code>    |

|                                                     |                                                                                                                                        |                                                 |
|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>--ui</code>                                   | Habilita interface gráfica (Fyne)                                                                                                      | <code>--ui</code>                               |
| <code>--headless</code>                             | Desabilita UI (padrão)                                                                                                                 | <code>--headless</code>                         |
| <code>--db</code><br><code>&lt;backend&gt;</code>   | Backend de banco: sqlite (padrão)                                                                                                      | <code>--db sqlite</code>                        |
| <code>--db-path</code><br><code>&lt;path&gt;</code> | Caminho do banco SQLite                                                                                                                | <code>--db-path</code><br><code>dados.db</code> |
| <code>--port &lt;n&gt;</code>                       | Porta do modo web ( <code>serve</code> )                                                                                               | <code>--port 9000</code>                        |
| <code>-w, --watch</code>                            | Hot reload no modo web ( <code>serve</code> )                                                                                          | <code>--watch</code>                            |
| <code>-o &lt;file&gt;</code>                        | Arquivo de saída (compile/build)                                                                                                       | <code>-o</code><br><code>saida.bytecode</code>  |
| <code>--gui</code>                                  | <code>build</code> : fixa app desktop (janela Fyne sempre; no Windows sem subsistema console) — obrigatório para <code>MSDIALOG</code> | <code>--gui</code>                              |

## Banco de dados compartilhado

Sem `--db-path`, o runtime resolve o banco na mesma ordem de todas as ferramentas AdvPP: variável `ADVPP_DB` → config `~/.advpp/advpp_config.json` (só se esse arquivo já existir) → banco local `./advpp.db` no diretório de trabalho atual, criado automaticamente. Assim `DBSelectArea` / `DBSeek` / `RecCount` / `RetSqlName` etc. já funcionam sem nenhuma configuração prévia, e enxergam exatamente o mesmo banco que o AdvEditor (rodado no mesmo diretório) usa para criar/editar tabelas.

## Encoding CP-1252

Fontes em CP-1252 (padrão Protheus) são detectados e convertidos automaticamente para UTF-8 por conversor interno 100% Go — não há dependência do `iconv` e o comportamento é idêntico em Linux, Windows e macOS.

## Sintaxe AdvPL

### Estrutura Básica

```
// Comentário de linha
/* Comentário de bloco */

#include "totvs.ch"

function NomeFuncao(param1, param2)
    local nVar1 := 0
    local cVar2 := ""

    // Código da função
    nVar1 := param1 + param2

return nVar1
```

### User Functions

```
user function NomeUF(param1)
    local nResult := 0

    // Código da user function
    nResult := param1 * 2
```

```
return nResult
```

## Variáveis

```
// Variáveis locais
local nNumero := 0
local cTexto := "Hello"
local dData := Date()
local lLogico := .T.
local aArray := {}

// Variáveis privadas
private nPrivado := 0

// Variáveis públicas
public nPublico := 0
```

## Estruturas de Controle

```
// If/Else
if nValor > 10
    Alert("Maior que 10")
else
    Alert("Menor ou igual a 10")
endif

// Do While
do while nContador < 10
    nContador++
enddo

// For
for nI := 1 to 10
    Alert(Str(nI))
next nI

// Case
do case
case nOpcao == 1
    Alert("Opção 1")
case nOpcao == 2
    Alert("Opção 2")
otherwise
    Alert("Outra opção")
endcase
```

## Funções de Manipulação de Dados

```
// Database
dbSelectArea("SA1")
dbSeek("001")
dbSkip()
dbGoTop()
dbGoBottom()

// Queries
TCQuery("SELECT * FROM SA1 WHERE A1_FILIAL = '01'")
```

## Comandos customizados ( `#xcommand` / `#command` / `#xtranslate` / `#translate` )

O pré-processor expande de verdade comandos customizados definidos em

headers `.ch`, no estilo do pré-processador do Clipper — não é só sintaxe reconhecida e descartada.

```
#command STORE HEADER <cA> T0 <aH> [FOR <for>] ;
=> SX3->(dbSetOrder(1)) ;
    ; SX3->(DBEval({|| AaDd(<aH>,{SX3->X3_CAMPO})}, {|| PLSCHKNIV(<for>) })))

STORE HEADER "SA1" T0 aHead FOR MeuFiltro()
// expande para: SX3->(dbSetOrder(1)) ; SX3->(DBEval({|| AaDd(aHead,{SX3->X3_CAMPO})}, {|| PLSCHKNIV(MeuFiltr
```

Sintaxe do padrão (lado esquerdo do `=>`)

| Elemento                            | Significado                                                                               |
|-------------------------------------|-------------------------------------------------------------------------------------------|
| <code>PALAVRA</code>                | Literal, casado sem diferenciar maiúsculas/minúsculas                                     |
| <code>&lt;nome&gt;</code>           | Captura o que aparecer até o próximo literal esperado                                     |
| <code>&lt;nome, ...&gt;</code>      | Captura uma lista (texto cru, vírgulas inclusas)                                          |
| <code>[...]</code>                  | Grupo opcional — só é tentado se o primeiro literal dentro dele aparecer na posição atual |
| <code>[&lt;nome:LITERAL&gt;]</code> | Marcador de flag booleana (vira <code>&lt;.nome.&gt;</code> no resultado)                 |

Sintaxe do resultado (lado direito do `=>`)

| Elemento                    | Significado                                                                                                      |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>&lt;nome&gt;</code>   | Substitui pelo texto capturado (vazio se a cláusula estava ausente)                                              |
| <code>&lt;{nome}&gt;</code> | Vira <code>{   texto}</code> se capturado, <code>NIL</code> se ausente — para condições <code>FOR / WHILE</code> |
| <code>&lt;.nome.&gt;</code> | <code>.T.</code> se capturado/flag presente, <code>.F.</code> caso contrário                                     |
| <code>\[ / \]</code>        | Colchete literal no texto de saída                                                                               |

Definições podem se espalhar por várias linhas físicas via continuação com `;` no final — mesma convenção do código comum.

Limitações conhecidas

- Casamento só no início da linha/statement (não no meio de uma expressão maior).
- Tokenização própria (não usa o lexer completo) — cobre bem os casos reais mais comuns, mas construções muito incomuns dentro dos argumentos capturados podem não ser preservadas perfeitamente.

Sintaxe TLPP

Classes

```
class MinhaClasse
  private:
    nValor := 0
    cNome := ""

  public:
    method New(nValor, cNome) constructor
    method GetValor()
    method SetValor(nValor)
```

```

        method GetNome()
        method SetNome(cNome)
endclass

method New(nValor, cNome) class MinhaClasse
    ::nValor := nValor
    ::cNome := cNome
return self

method GetValor() class MinhaClasse
return ::nValor

method SetValor(nValor) class MinhaClasse
    ::nValor := nValor
return

```

## Interfaces

```

interface IMinhaInterface
    method Metodo1()
    method Metodo2()
endinterface

class MinhaClasse implements IMinhaInterface
    public:
        method Metodo1()
        method Metodo2()
endclass

```

**Limitação conhecida:** `interface...endinterface` é parseado como declaração real ( `InterfaceDecl` ), com seus métodos). Já a cláusula `implements` na `class` é aceita apenas sintaticamente — o AdvPP **não** valida que a classe de fato implementa os métodos da interface, nem falha a compilação se algum método faltar.

## Operadores

```

class Numero
    private:
        nValor := 0

    public:
        method New(nValor) constructor
        method operator+(obj) // Sobrecarga de +
        method operator-(obj) // Sobrecarga de -
endclass

method operator+(obj) class Numero
    local nResult := Numero()
    nResult:SetValor(::nValor + obj:GetValor())
return nResult

```

## Try/Catch

```

try
    // Código que pode gerar erro
    nResult := 10 / 0
catch e
    // Tratamento de erro
    Alert("Erro: " + e:Message())
finally
    // Sempre executado

```

```
    Alert("Fim")
endtry
```

## JSON Inline

```
local jData := {
    "nome": "João",
    "idade": 30,
    "ativo": true
}

local cNome := jData["nome"]
local nIdade := jData["idade"]
```

## Namespaces

```
namespace MeuNamespace

class MinhaClasse
    // ...
endclass

endnamespace

// Uso
local obj := MeuNamespace.MinhaClasse()
```

## Tipagem

```
// Tipos explícitos
function Soma(n1: numeric, n2: numeric): numeric
return n1 + n2

// Inferência de tipos
local nValor := 10 // numeric
local cTexto := "Hello" // character
```

## Parâmetros Nomeados

```
function MinhaFuncao(p1, p2, p3)
    // ...
return

// Chamada com parâmetros nomeados
MinhaFuncao(p3=10, p1=20, p2=30)
```

## Erros Comuns de Compilação

### Erros de Sintaxe

Erro: Esperado ';'

```
// Errado
local nVar := 0

// Correto
local nVar := 0;
```

Erro: Parênteses não fechados

---

```
// Errado
if nValor > 10
    Alert("Maior")

// Correto
if nValor > 10
    Alert("Maior")
endif
```

## Erros de Tipo

Erro: Tipo incompatível

```
// Errado
local nNumero := "texto" // string em vez de número

// Correto
local nNumero := 10
local cTexto := "texto"
```

## Erros de Escopo

Erro: Variável não declarada

```
// Errado
nValor := 10 // variável não declarada

// Correto
local nValor := 10
```

## Erros de Include

Erro: Arquivo não encontrado

```
// Errado
#include "arquivo_inexistente.ch"

// Correto
#include "totvs.ch"
```

## Performance

O compilador é otimizado para fontes reais do Protheus: um arquivo de ~574KB compila em ~0,1s e um lote de 300 fontes reais é verificado em ~1,2s com `advplc check` paralelo (1 worker por CPU).

Não há níveis de otimização configuráveis — o bytecode gerado é único.

## Multi-thread no runtime

### StartJob

```
// StartJob(cFunc, cEnv, lWait, params...)
StartJob("U_Worker", "ENV", .F., "parametro") // assíncrono
StartJob("U_Worker", "ENV", .T., "parametro") // síncrono (bloqueia)
```

Cada job roda em uma VM isolada (memória própria, como um work process do Protheus) com sua própria conexão ao banco SQLite compartilhado. Jobs assíncronos pendentes são aguardados antes de o processo encerrar.

### FWGridProcess

Processamento em grid com pool de threads (ver documentação TDN):

```
oGrid := FWGridProcess():New("ROTINA", "Titulo", "Descricao", Nil, "", "U_GridWk", .T.)
oGrid:SetThreadGrid(4) // pool de 4 threads
For nX := 1 To 100
  If !oGrid:CallExecute(nX) // despacha p/ o pool (backpressure)
    Exit // .F. = StopExecute() foi chamado
  EndIf
Next nX
oGrid:Activate() // espera as threads terminarem
If oGrid:IsFinished()
  ConOut("ok")
EndIf
```

Métodos suportados: `New`, `Activate`, `Execute`, `DeActivate`, `SetThreadGrid`, `SetMaxThreadGrid`, `CallExecute`, `StopExecute`, `IsFinished`, `SetAbort`, `SetAfterExecute`, `SetMeters`, `SetMaxMeter`, `SetIncMeter`, `SetNoParam`, `SaveLog`, `GetLastLog`.

A interface gráfica de configuração do Protheus não é reproduzida (runtime headless); a semântica de processamento é completa.

### Motor de inferência LLM (classe `LLM`)

O AdvPP embute um motor de inferência para modelos de linguagem quantizados em `I2_S` (pesos ternários -1/0/+1, formato BitNet) ou, desde v4.0.0, em `Q4_K/Q6_K` (ex.: conversões `Q4_K_M` como MiniCPM), escrito inteiramente em Go — sem CGO, sem `llama.cpp`, sem dependências de terceiros. Compila e roda de forma idêntica em Linux, Windows e macOS (amd64/arm64).

### Exemplo

```
User Function LlmDemo()
  Local oLLM
  Local cModelo := "/caminho/Falcon3-3B-Instruct-1.58bit/ggml-model-i2_s.gguf"

  oLLM := LLM():New(cModelo)
  ConOut(oLLM:Generate("The capital of France is", 6, 0))
  oLLM:Close()
Return
```

### Métodos

| Método                | Parâmetros                                                                       | Retorno                    | Descrição                                                                                                                                                                                                                                    |
|-----------------------|----------------------------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>New</code>      | <code>cCaminhoGGUF</code>                                                        | <code>Object</code> (self) | Carrega o modelo e o tokenizer. Pode levar alguns segundos (o arquivo GGUF tem tipicamente 1-2GB).                                                                                                                                           |
| <code>Generate</code> | <code>cPrompt</code> ,<br><code>nMaxTokens</code> ,<br><code>nTemperatura</code> | <code>Character</code>     | Roda o prompt pela rede e amostra até <code>nMaxTokens</code> novos tokens. <code>nTemperatura&lt;=0</code> faz amostragem gulosa (greedy, determinística); a chamada <b>bloqueia</b> até terminar ou encontrar o token de fim de sequência. |
| <code>Tokenize</code> | <code>cTexto</code>                                                              | <code>Array</code>         | Retorna os token ids (números) do texto, via BPE byte-level.                                                                                                                                                                                 |
| <code>Decode</code>   | <code>aTokens</code>                                                             | <code>Character</code>     | Converte um array de token ids de volta em texto.                                                                                                                                                                                            |
| <code>Close</code>    | —                                                                                | <code>Nil</code>           | Libera o arquivo do modelo.                                                                                                                                                                                                                  |

### Arquitetura suportada

Arquitetura GGUF `general.architecture = "llama"` (é o caso do BitNet original convertido para essa arquitetura, de conversões como o `Falcon3-3B-Instruct-1.58bit`, e de conversões `Q4_K_M` / `Q6_K` como o MiniCPM5-2B) ou `"minicpm"` (com as escalas muP próprias, quando presentes no GGUF). Pesos de camada em `I2_S`, `F16`, `Q4_K` ou `Q6_K` — decidido pelo tipo real de cada tensor, não precisa ser uniforme no arquivo (ex.: camadas em `Q4_K` com `token_embd` / `output` em `Q6_K`, como o MiniCPM5-2B-`Q4_K_M`). Não suporta (ainda) a arquitetura customizada `bitnet-b1.58` (que tem normas extras "SubLN" no grafo), outras arquiteturas (Qwen, Mistral, modelos com sliding-window attention, etc.), F32, nem os demais k-quants (`Q2_K`/`Q3_K`/`Q5_K`/`Q8_K` etc.) — ver `pkg/llm/model.go`.

## Desempenho e SIMD

O kernel do produto escalar ternário usa **AVX2** em CPUs amd64 que o suportam (detecção via CPUID em tempo de execução — sem AVX2, ou fora de amd64, cai automaticamente para um caminho escalar puro em Go, idêntico em resultado, só mais lento). Matmuls e atenção são paralelizados por faixa de linhas/cabeças via goroutines (`runtime.GOMAXPROCS`). Referência: ~5s/token no Falcon3-3B-1.58bit em 8 núcleos com AVX2.

## Validação

O motor foi validado **token a token** contra o `llama.cpp` de referência (mesmo algoritmo do BitNet oficial) — ver `pkg/llm/validate_test.go`. Rode `go test ./pkg/llm/...` (sem `-short`) para reproduzir, desde que tenha o modelo e um binário `llama-cli` de referência disponíveis localmente.

## Limitações conhecidas

- Sem streaming: `Generate` só devolve o texto completo ao final.
- Pré-tokenizador simplificado: não replica o split dígito-a-dígito específico do pré-tokenizador "falcon3" do llama.cpp (só afeta números com mais de um dígito).
- Uma sessão (`Context`) por objeto `LLM`; sem suporte a múltiplas sequências simultâneas na mesma chamada.

## Servidor MCP (classe `MCPServer`)

O AdvPP fala **MCP (Model Context Protocol)** nativamente — um servidor que expõe funções AdvPL/TLPP como "tools" que qualquer cliente MCP (Claude, outros agentes de IA) pode listar e chamar via JSON-RPC 2.0 sobre stdio. Diferente do suporte a REST (`WSRESTFUL` / `WSSERVICE` / `@Get` / `@Post`), que hoje só **reconhece a sintaxe** e a descarta (sem subir servidor HTTP nem despachar nada — ver seção de Sintaxe TLPP), o `MCPServer` **executa de verdade**.

## Exemplo

```
User Function McpDemo()
    Local oMCP := MCPServer():New("meu-servidor", "1.0.0")

    oMCP:AddTool("soma", "Soma dois números", ;
        '{"type":"object","properties":{"a":{"type":"number"},"b":{"type":"number"}},"required":["a","b"]}',
        "ToolSoma")

    oMCP:Serve() // bloqueia lendo stdin / escrevendo stdout
Return
```

```
User Function ToolSoma(oArgs)
Return cValToChar(oArgs:A + oArgs:B)
```

Rode com `advplc run mcp_demo.prw` — não precisa de nenhum comando novo de CLI. O processo vira um servidor MCP: qualquer cliente que fale o protocolo (ex.: via `stdio_client` do SDK oficial) pode se conectar via stdin/stdout do processo.

Métodos

| Método  | Parâmetros                                           | Descrição                                                                                                                                                                                                                                                                                                          |
|---------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| New     | cNome,<br>cVersao                                    | Cria o servidor (nome/versão aparecem no handshake <code>initialize</code> )                                                                                                                                                                                                                                       |
| AddTool | cNome,<br>cDescricao,<br>cSchemaJSON,<br>cNomeFuncao | Registra uma tool. <code>cSchemaJSON</code> é o JSON Schema dos parâmetros (pode ser "" para aceitar qualquer objeto). <code>cNomeFuncao</code> é o nome de uma User Function que recebe <b>um objeto</b> com os argumentos da chamada ( <code>oArgs:CAMPO</code> , em maiúsculas) e devolve o texto do resultado. |
| Serve   | —                                                    | Sobe o loop de mensagens JSON-RPC sobre stdio. <b>Bloqueia</b> até o cliente fechar a conexão (EOF no stdin). Redireciona <code>ConOut</code> /console para stderr automaticamente, para não misturar saída de depuração com as mensagens do protocolo no stdout.                                                  |

Isolamento de execução

Cada `tools/call` roda a função registrada em uma **VM isolada** (mesmo mecanismo do `StartJob`) — necessário porque `Serve()` já está em execução no meio da VM principal quando uma chamada chega; invocar a função direto na mesma VM corromperia a pilha de chamadas em andamento.

Protocolo suportado

`initialize` , `notifications/initialized` , `tools/list` , `tools/call` , `ping` — o essencial para expor tools. Não implementa `resources/*` , `prompts/*` nem `sampling/*` (extensões do protocolo fora do escopo de "tools", que é o caso de uso mais comum).

Validação

Testado com o **SDK oficial em Python do MCP** ( `stdio_client` + `ClientSession` ), não só com mensagens JSON-RPC feitas à mão — ver `cmd/advplc/mcp_integration_test.go` .

gRPC embarcado (classes `GRPCServer` e `tGrpc` )

O AdvPP embarca o framework gRPC real ( `google.golang.org/grpc` ) — tanto para **expor** quanto para **consumir** serviços gRPC, com HTTP/2 e protobuf reais na wire.

Servidor — `GRPCServer`

```
User Function Ping(oParams)
    Local oResp := JsonObject():New()
    oResp["pong"] := "hello " + oParams["name"]
Return oResp

User Function GrpcDemo()
    Local oServer := GRPCServer():New()
    oServer.AddMethod("Echo", "Ping", "Ping") // serviço, método, função
    oServer.Serve(50051) // bloqueia
```

## Return

AdvPL/TLPP não tem como declarar tipos `.proto` estaticamente — por isso `GRPCServer` define em runtime um único tipo de mensagem genérico (`message JsonEnvelope { string json = 1; }`) reaproveitado por toda RPC, com **Server Reflection habilitada**: qualquer cliente gRPC real (`grpcurl`, um cliente `protoc`, ou o `tGrpc` deste mesmo compilador) descobre e chama o serviço sem precisar de nenhum `.proto` gerado em tempo de compilação. Os parâmetros chegam como `JsonObject` (`oParams["campo"]`), acesso por colchete — case-sensitive, semântica JSON) e o retorno da função vira o corpo JSON da resposta.

| Método                                             | Descrição                                                |
|----------------------------------------------------|----------------------------------------------------------|
| <code>New()</code>                                 | Cria o servidor                                          |
| <code>AddMethod(cServico, cMetodo, cFuncao)</code> | Registra a RPC (antes de <code>Serve()</code> )          |
| <code>Serve([nPorta])</code>                       | Sobe o servidor (default <code>:50051</code> ), bloqueia |
| <code>Shutdown()</code>                            | Encerra graciosamente                                    |

## Cliente — `tGrpc`

```
User Function GrpcClientDemo()  
  Local oClient := tGrpc():New("smartlink.proto", "localhost", 50051)  
  If oClient:isRunning()  
    oClient:MsgContent := '{"name":"AdvPP"}'  
    oClient:sendMessage()  
  EndIf  
Return
```

Abre uma conexão HTTP/2 real e descobre em runtime, via gRPC Server Reflection, quais serviços/métodos o servidor alvo expõe — sem precisar de `.proto` local. Testado ponta a ponta contra servidores gRPC reais (inclusive `tGrpc` ↔ `GRPCServer` entre si). Detalhe completo em `docs/tdn-known-limitations.md`.

## Integração com IDE

### Compilação via IDE

O AdvPP IDE integra o AdvPlc automaticamente:

1. Abra o arquivo no editor
2. Pressione **F5** para compilar
3. O resultado aparece no terminal
4. Erros são destacados no editor

### Configuração do Compilador

Configure o compilador no IDE:

1. Menu **Ferramentas** → **Configurações** → **Compilação**
2. Configure:
  - Caminho do compilador
  - Flags de compilação
  - Diretórios de include
  - Nível de otimização

## Projetos

### Arquivo de Projeto

Crie um arquivo `advpl-project.json`:

```
{
  "name": "Meu Projeto",
  "version": "1.0.0",
  "target": "tlpp",
  "optimization": 2,
  "includes": [
    "./include"
  ],
  "defines": [
    "DEBUG"
  ],
  "sources": [
    "./src"
  ],
  "output": "./build"
}
```

### Compilar Projeto

```
advplc -p advpl-project.json
```

## Boas Práticas

### Nomenclatura

- **Funções:** Use `PascalCase` para funções
- **Variáveis:** Use `camelCase` para variáveis
- **Constantes:** Use `UPPER_CASE` para constantes
- **Classes:** Use `PascalCase` para classes

### Organização

```
projeto/
├─ src/           # Arquivos fonte
│  └─ main.prw
│  └─ functions.prw
├─ include/       # Arquivos de cabeçalho
│  └─ myheader.ch
├─ build/         # Arquivos compilados
└─ advpl-project.json
```

### Comentários

```
// Comentário de linha - use para explicações curtas

/*
 * Comentário de bloco
 * Use para explicações detalhadas
 */

/**
 * Documentação de função
 * @param nParam1 Descrição do parâmetro
 * @return Descrição do retorno
 */
```

```
function MinhaFuncao(nParam1)
    // ...
    return
```

## Validação

```
// Valide entradas
if Empty(cNome)
    Alert("Nome obrigatório")
    return .F.
endif

// Valide tipos
if ValType(nValor) != "N"
    Alert("Valor deve ser numérico")
    return .F.
endif
```

## Solução de Problemas

### Erro: Arquivo não encontrado

**Causa:** Caminho do arquivo incorreto

**Solução:**

```
# Verifique se o arquivo existe
ls -la arquivo.prw

# Use caminho absoluto
advplc /caminho/completo/arquivo.prw
```

### Erro: Include não encontrado

**Causa:** Diretório de include não configurado

**Solução:**

```
# Adicione diretório de include
advplc -I ./include arquivo.prw
```

### Erro: Memória insuficiente

**Causa:** Arquivo muito grande

**Solução:**

```
# Compile arquivos individualmente
advplc arquivo1.prw
advplc arquivo2.prw
```

### Erro: Sintaxe inválida

**Causa:** Erro de sintaxe no código

**Solução:**

```
# Use modo verboso para detalhes
advplc -v arquivo.prw

# Verifique linha indicada no erro
```

## Performance

### Tempo de Compilação

- **Arquivo pequeno** (< 100 linhas): < 50ms
- **Arquivo médio** (100-1000 linhas): < 200ms
- **Arquivo grande** (> 1000 linhas): < 1s

### Tamanho do Bytecode

- **Arquivo pequeno**: ~1KB
- **Arquivo médio**: ~10KB
- **Arquivo grande**: ~100KB

### Otimizações de Performance

- Use `-O2` para melhor equilíbrio
- Compile apenas arquivos modificados
- Use include paths corretos
- Evite includes desnecessários

## Exemplos Completos

### Exemplo 1: Hello World

```
#include "totvs.ch"

function Hello()
    Alert("Hello, World!")
return
```

### Exemplo 2: Função Matemática

```
function Soma(n1, n2)
    local nResult := 0

    nResult := n1 + n2

return nResult
```

### Exemplo 3: Manipulação de Banco

```
function ConsultaCliente(cCod)
    local cNome := ""

    dbSelectArea("SA1")
    dbSeek(cCod)

    if Found()
        cNome := SA1->A1_NOME
    endif

return cNome
```

### Exemplo 4: Classe TLPP

```
class Pessoa
    private:
        cNome := ""
        nIdade := 0
```

```

    public:
        method New(cNome, nIdade) constructor
        method GetNome()
        method SetNome(cNome)
        method GetIdade()
        method SetIdade(nIdade)
endclass

method New(cNome, nIdade) class Pessoa
    ::cNome := cNome
    ::nIdade := nIdade
return self

method GetNome() class Pessoa
return ::cNome

method SetNome(cNome) class Pessoa
    ::cNome := cNome
return

```

## Referência de Comandos

### Comandos Rápidos

```

# Compilar arquivo
advplc arquivo.prw

# Compilar com saída específica
advplc -o output.bytecode arquivo.prw

# Compilar diretório
advplc -d ./src

# Compilar com otimização
advplc -O2 arquivo.prw

# Compilar com debug
advplc -g arquivo.prw

# Compilar projeto
advplc -p advpl-project.json

# Mostrar versão
advplc --version

# Mostrar ajuda
advplc --help

```

## Conclusão

O AdvPlc é um compilador poderoso e flexível para AdvPL/TLPP, suportando tanto a sintaxe clássica quanto as extensões modernas. Com este manual, você deve ser capaz de compilar código fonte, otimizar performance e resolver problemas comuns de compilação.

Para mais informações, visite a documentação oficial em <https://github.com/peder1981/AdvPP>.